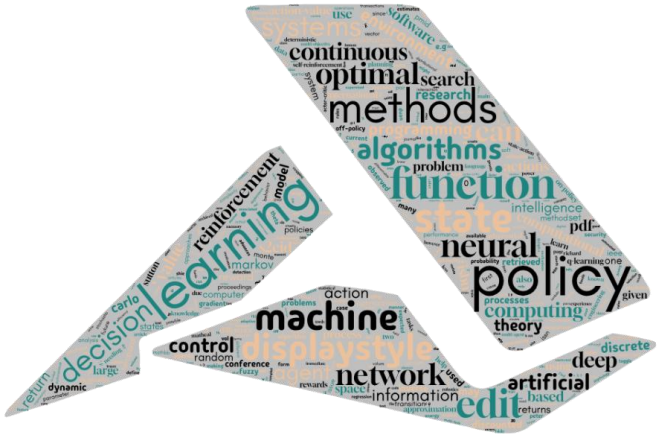


Reinforcement Learning Algorithms and Applications

Chapter 9. Double Deep Q Network for CartPole

Introduction to DQN and Double DQN

- **Deep Q-Network (DQN)** uses a single neural network to estimate Q-values and select actions.
- **Double DQN (DDQN)** addresses the **overestimation bias** problem in standard DQN.
- DDQN uses **two networks**: policy network for action selection and target network for value estimation.
- Both algorithms use the same **network architecture** and **replay memory mechanism**.
- The primary difference lies in the **Q-value calculation** during training.
- DDQN provides more **stable** and **accurate** Q-value estimates.
- Both implementations support **CartPole-v1 environment** with identical training configurations.

Core Algorithm Differences

- The goa of DQN is to minimize the TD error via the following loss function:
  - $L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q_{target}(s', a') - Q_{policy}(s, a)\right)^2\right]$
- **DQN Bellman Equation:**
  - $y = r + \gamma \max_{a'} Q_{target}(s', a')$
- **Double DQN Bellman Equation:**
  - $y = r + \gamma\, Q_{target}(s', \arg\max_{a'} Q_{policy}(s', a'))$
- **DQN** uses the same network for both action selection and value estimation.
- **DDQN** separates action selection (policy network) and value evaluation (target network).
- This separation **reduces overestimation bias** that occurs in standard Q-learning.
- DDQN prevents the algorithm from being overly optimistic about action values.

Project Structure Modifications

- **Model directory** changed from "dqn_models" to "ddqn_models".
- **Interface titles** updated to reflect Double DQN usage.
- **Training mode** displays "TRAINING MODE (Double DQN)" instead of standard DQN.
- **Training messages** specify "Starting Double DQN training" for clarity.
- **Bellman equation comments** updated to reflect the double Q-learning approach.
- All other **hyperparameters** and **training configurations** remain identical.
- **Network architecture** (DQN class) stays completely unchanged.

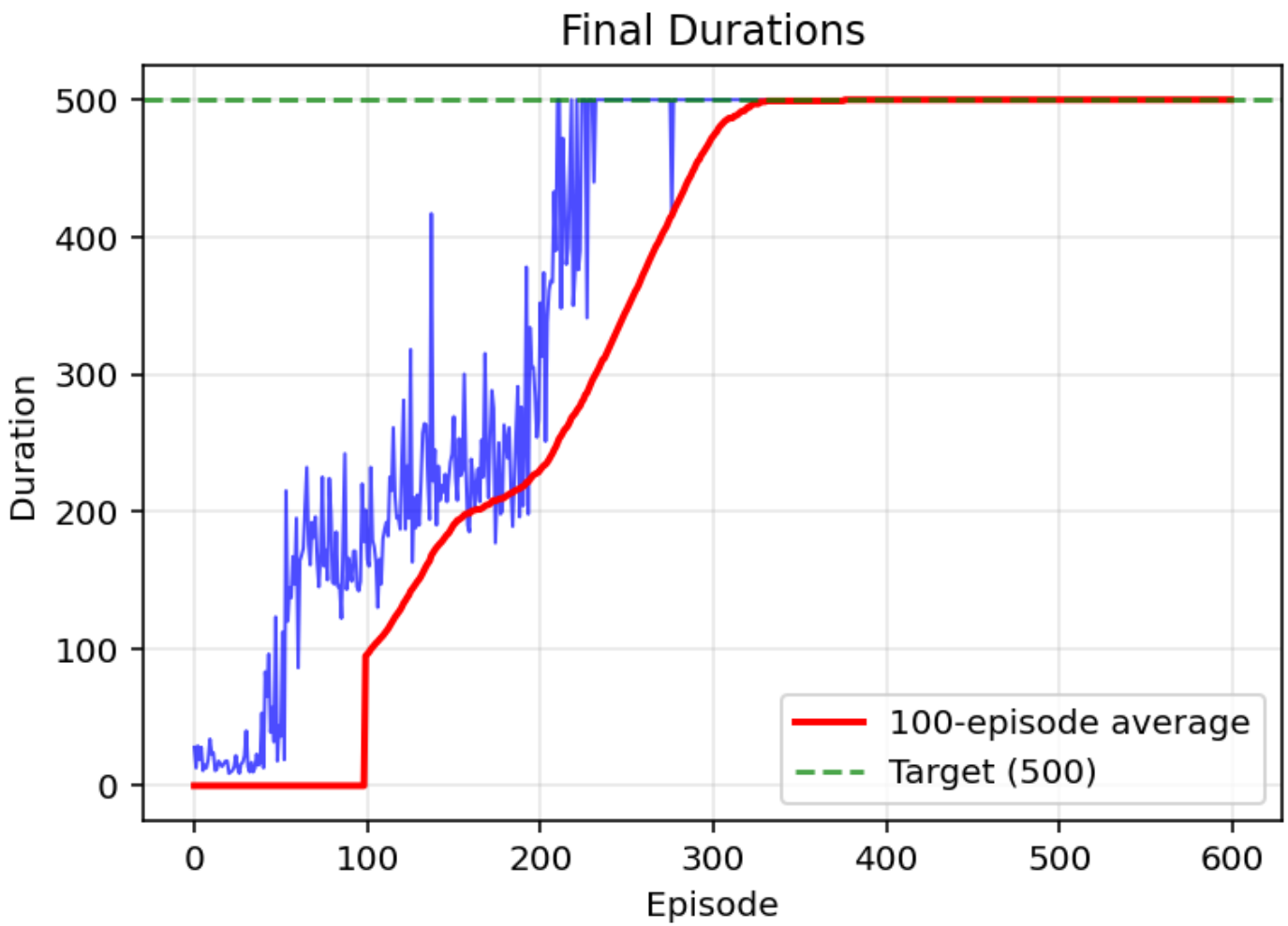
Benefits of Double DQN

- **Reduces overestimation bias** inherent in standard Q-learning algorithms.
- Provides more **accurate Q-value estimates** during training.
- Leads to **better policy performance** and more stable learning.
- **Faster convergence** to optimal policies in many environments.
- **Improved sample efficiency** compared to standard DQN.
- **Minimal computational overhead** - only requires changing the target calculation.
- **Easy to implement** - requires only a few lines of code modification.

Practical Implementation Summary

- **One core algorithm change** in the optimize_model() function.
- **Four interface text updates** for user clarity and identification.
- **One directory name change** to separate model storage.
- **Same network architecture** and **training pipeline** used for both methods.
- **Identical hyperparameters**: learning rate, batch size, epsilon decay, etc.
- **Compatible environments**: Both work with CartPole-v1 and other Gym environments.
- **Backward compatibility**: DDQN models can be loaded and demonstrated using the same framework.

Training Result



Reference

- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.